The primitives' page: the join and the visible write

Programming Paradigms · Lab 7 · session 7: multi-threading

Ten questions, each in the form *given ..., say ...*, on the two primitives of the lecture that are not structures and are not built: the join, on which `Workers` rests, and the visible write, on which its flag rests. Questions 6, 7, 8, 1 and 5 are taken up in the lab, in that order, between the structures; the rest over the week. Each question gives the form of its answer, so that a right answer can be told from a wrong one at home; the answers are on the teacher's copy. Nothing is handed in: the lab collects nothing.

Three phrases the lab uses as names, from session 6: *not at the same time*, two threads are never both inside the part that changes shared data; *not before*, a thread does not act until another has acted; *visibility*, a write by one thread is seen by a read on another. The lock, the atomic instruction and the per-thread copy settle the first; the join, the barrier and the monitor's wait settle the second, and each of them settles the third as well: what was written before the join, the arrival, or the release of the monitor is seen after it. The volatile write settles the third alone: the word written, and everything written before it, are seen by the thread that reads the word, and nothing else; it forbids no overlap and makes no two fields change as one.

The join

The guarantee.

The join. After `t.join()` returns, `t` has ended, and everything `t` wrote is visible to the thread that joined it; `t.start()` is its mirror: everything written before the start is visible to `t`.

Question 1

Given a worker that writes its result into `counts[id]`, and a main thread that reads the array, say what main may read in `counts[id]` before the join, after the join, and after joining only the other workers.

Answer form: for each, one of: the written value; anything, the old value included.

Question 2

Given three workers that each write their own slot of `counts`, joined in the order 2, 0, 1, say whether the order of the joins changes what main sees after the three.

Answer form: yes or no, and the sentence of the guarantee that says so.

Question 3

Given two workers that both write one plain field `last`, both joined, say what the join guarantees about the field and what it does not.

Answer form: what main sees, and what is left open.

Question 4

Given the guarantee above and a thread `t` that was made with `new Thread(r)` but never started, say what `t.join()` guarantees and what it does not.

Answer form: when the join returns, and what it says about `r`'s writes.

Question 5

Given the guarantee of the join above, say which of the three phrases it settles: *not at the same time*, *not before*, visibility.

Answer form: one or more of the three.

The visible write

The guarantee.

The visible write. All reads and writes of volatile fields in a program fall in one order that every thread agrees on, consistent with each thread's own order; a write to a volatile field is seen by every later read of it in that order, on any thread, and everything written before that write is visible after that read. It does not make a read-modify-write one step, and does not make two fields change as one.

Question 6

Given the loop `while (!stop) { }` on a plain flag `stop`, written to `true` by another thread, say whether the loop may spin for ever and why the compiler is allowed to make it; then with `stop` declared `volatile`.

Answer form: may or may not, and the rule that allows it. The `Workers` check is this question run.

Question 7

Given a plain field `data` written by thread A before A sets the volatile flag `ready` to `true`, and a reader on thread B that sees `ready` true, say what B may read in `data`.

Answer form: the written value only, or anything.

Question 8

Given a `volatile int n`, two threads that each do `n++` a thousand times, and the value of `n` printed after both are joined, say whether it is always 2000, and if not, the range it may fall in and why `volatile` does not make it 2000.

Answer form: yes, or a range and a sentence.

Question 9

Given two volatile fields `x` and `y`, both 0, thread A doing `x = 1; r1 = y` and thread B doing `y = 1; r2 = x`, say whether `r1` and `r2` can both be 0; then with plain fields. Then run the given program, `java -cp out Reorder plain` and `java -cp out Reorder volatile`: with plain fields it may happen, and it did on the machine the lab was written on; a run where it does not happen proves nothing (rule 8). That is the lecture's discovery as a question.

Answer form: can or cannot, twice, each with its reason.

Question 10

Given four needs, a stop signal; a counter that many threads add to; a count that one thread writes and another reads after a join; two fields that must change together, say for each whether a visible flag, an atomic, a count per thread, or a lock is the right tool, and why the others are not.

Answer form: one tool per need.